

The Technology Stack

A plain-language tour of the tools used to build the mobile app, save data offline, synchronize with a server, and prepare for MobileNetV3 image recognition.

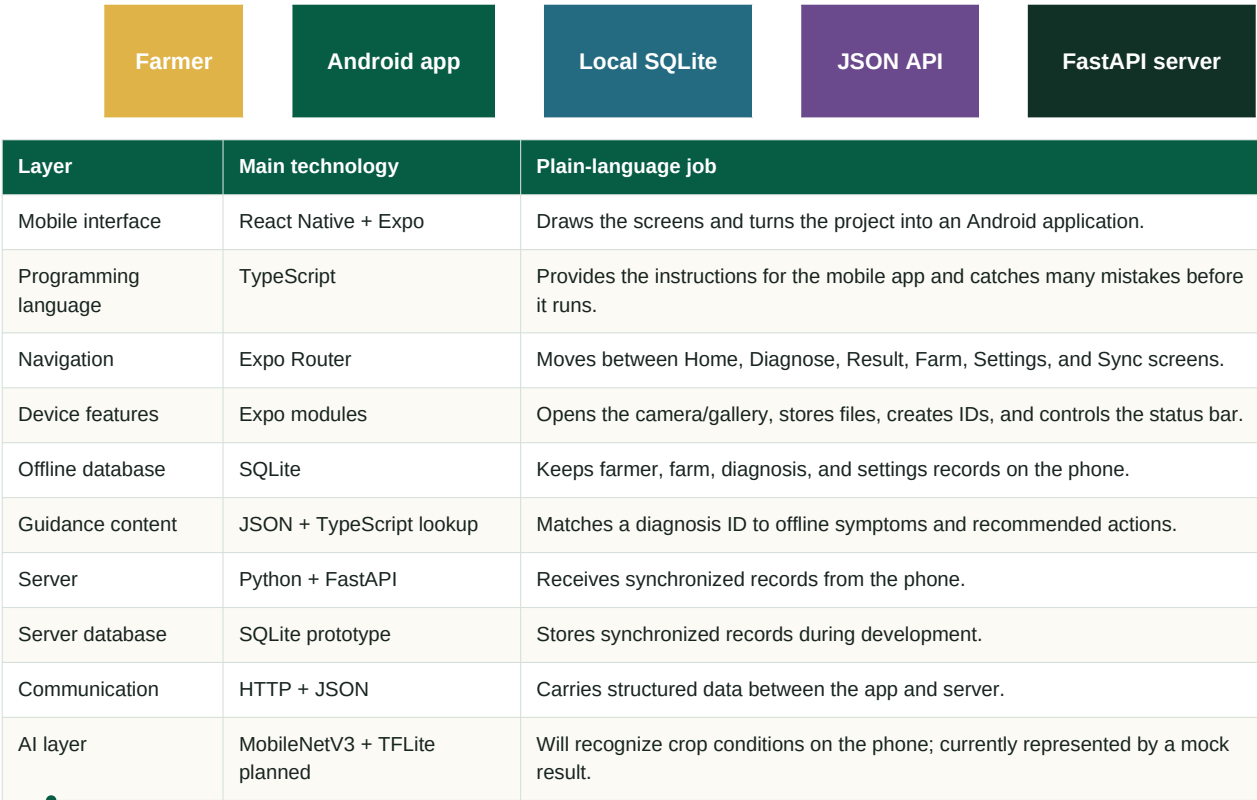
What is a technology stack?

It is the collection of programming languages, frameworks, databases, and tools that work together to make an application. Think of it as the app's construction materials and machinery.

This guide describes the current project state. MobileNetV3 is the planned image-recognition layer; the running app still uses a mock classifier.

1. The whole system at a glance

LimaDRC is an **offline-first Android app**. A farmer can take a crop photo and save an observation even when the internet is unavailable. When a connection returns, the app can send pending records to a backend server.



A useful mental model
The mobile app is the field notebook, SQLite is its local filing cabinet, the API is the delivery service, and the backend is the central office.

2. The mobile app

The mobile side is written once and rendered as a native-style application. Most of its source lives under **mobile/src/**.

React Native 0.86.3

React Native lets developers build phone screens using React concepts instead of writing every Android screen directly in Kotlin or Java. Components such as text, buttons, images, and cards are combined to make the interface.

React 19.2.3

React organizes the interface into reusable pieces called **components**. It also updates the screen when information changes. For example, selecting a photo changes the Diagnose screen from an empty placeholder to an image preview.

Expo SDK 57

Expo provides a managed toolbox around React Native. It handles much of the Android setup and supplies ready-made modules for common phone features. The project uses Expo modules for:

- **expo-image-picker**: opens the camera or photo library.
- **expo-file-system**: copies crop photographs into private app storage.
- **expo-sqlite**: provides the local database.
- **expo-crypto**: creates unique IDs for records and saved images.
- **expo-status-bar**: controls the phone's status-bar appearance.

TypeScript 6

TypeScript is JavaScript with extra rules about the shape of data. If a diagnosis is supposed to include a crop, diagnosis ID, and confidence number, TypeScript can warn the developer when one is missing or has the wrong type.

```
type DiagnosisResult = {
  diagnosis_id: string;
  crop: string;
  confidence: number;
};
```

Why this combination?

React Native supplies the mobile interface, Expo simplifies device integration, and TypeScript makes the data contracts easier to understand and safer to change.

3. Screens, navigation, and development tools

Expo Router

Expo Router uses files to define screens. A file in **mobile/src/app/** becomes a route the user can visit.

File	Screen
index.tsx	Home
diagnose.tsx	Choose or take a crop photo
result.tsx	Show the diagnosis and local guidance
farm.tsx	Farmer, farm, and saved observation information
sync.tsx	Backend address, connection test, and synchronization status
settings.tsx	Language, notification, and audio-guidance preferences
onboarding.tsx	First-run introduction
_layout.tsx	Shared navigation frame and SQLite database provider

Metro

Metro is the development bundler used by React Native. It gathers the TypeScript and JavaScript modules, transforms them, and sends a development bundle to the emulator or phone. Metro is mainly a development tool; users do not run Metro after installing a production Android build.

Node.js and npm

Node.js runs the mobile development tools on the developer's computer. npm reads **mobile/package.json** and installs the exact libraries the app needs into **mobile/node_modules/**. Expo SDK 57 in this project expects Node.js 22.13 or newer.

Android emulator and ADB

Android Studio's emulator acts like a virtual Android phone. ADB, the Android Debug Bridge, lets development tools communicate with that emulator or a physical Android device, install builds, and read diagnostic logs.

Hermes

Hermes is the JavaScript engine commonly used by React Native on Android. It runs the app's JavaScript after Metro or the production build has prepared it.

4. Offline storage on the phone

Offline-first means the phone remains useful without a server connection. LimaDRC saves important information locally before attempting to synchronize it.

SQLite

SQLite is a small relational database stored inside the app's private Android sandbox. The database is named **limadrc.db** and is created by **mobile/src/storage/database.ts**. It contains four tables:

Table	What it stores
farmers	The farmer's profile, location fields, language, and synchronization state.
farms	The associated farm, primary crops, optional coordinates, and synchronization state.
observations	Photo location, crop, diagnosis ID, confidence, timestamp, and synchronization state.
settings	Small preferences such as the API address, last sync time, language, and feature settings.

PENDING and SYNCED

A new local record starts as **PENDING**. It stays on the phone even if the backend is unavailable. Only after the server confirms receipt does the app change that record to **SYNCED**. This is the basis of the offline queue.

```
Save locally -> PENDING -> try server -> server confirms -> SYNCED
|
+-- failure: remain PENDING and retry later
```

Private photo files

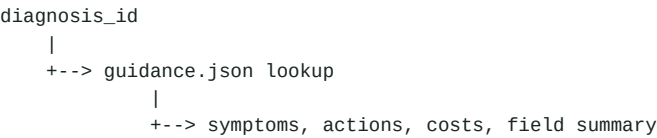
Selected crop images are copied into an **observation-images** directory in the app's private document storage. The database stores the image's local URI, which is the address of that file on the phone.

Current limitation
The prototype synchronizes the local image URI as text, not the image bytes. A production backend cannot open a private file path from someone else's phone. Production synchronization therefore needs a secure image-upload step or an explicit decision to keep photos only on-device.

5. Diagnosis guidance and the AI layer

Offline guidance catalog

The app currently bundles diagnosis guidance in **mobile/src/ml/guidance.json**. It contains structured names, symptoms, diagnosis messages, treatment actions, cost notes, and spoken-summary text. **guidance.ts** looks up a record using the diagnosis ID.



Because the JSON file is packaged with the app, guidance remains available without internet access. A validation script checks that its diagnosis IDs match the model label IDs.

Content safety

The guidance system is separate from the image model. Agricultural recommendations, chemical advice, availability, and costs must be reviewed and localized by appropriate DRC experts before production use.

Current mock classifier

The running prototype does not yet inspect the image with a trained model. **mockClassifier.ts** returns a fixed cassava mosaic result. This lets developers test the camera, result screen, saving, and synchronization flow before the real model is ready.

Planned MobileNetV3 and TensorFlow Lite

MobileNetV3Small is designed for devices with limited memory and processing power. The planned workflow is to train it using TensorFlow/Keras, export it to a **.tflite** file, and run that file directly on Android.

Stage	Technology	Role
Training	Python + TensorFlow/Keras	Teach MobileNetV3 using labeled crop photographs.
Export	TensorFlow Lite	Convert the trained model into a compact mobile format.
On-device inference	TFLite runtime adapter	Resize the photo, run the model, and receive 15 scores.
Interpretation	labels.json	Turn the highest score's position into a diagnosis ID.
Advice	guidance.json	Look up reviewed information for that diagnosis ID.

The model should return identification information only: **diagnosis_id**, **crop**, and **confidence**. It should not invent treatment recommendations.

6. The backend server

The backend is a separate program in the **backend/** directory. It receives records when the phone synchronizes.

Python

Python is the language used for the backend. It is readable, widely used for APIs and machine learning, and has a large collection of libraries.

FastAPI 0.142.2

FastAPI turns Python functions into web endpoints. It also uses Pydantic models to check incoming JSON. For example, confidence must be a number from 0 through 1.

Endpoint	Purpose
GET /health	Confirms that the prototype backend is running.
POST /farmers	Creates or updates a farmer record.
POST /farms	Creates or updates a farm record.
POST /observations	Creates or updates a diagnosis observation.
GET /observations	Returns stored observations for development and testing.

Uvicorn 0.54.0

Uvicorn is the server process that runs the FastAPI application and listens for network requests. FastAPI defines what the service does; Uvicorn keeps it running and reachable.

Backend SQLite

The prototype backend writes to **backend/database/limadrc.db**. It uses Python's built-in sqlite3 module. Writes are idempotent by local ID, meaning the phone can safely retry a record without intentionally creating a duplicate.

Prototype versus production

SQLite is excellent for development and small single-server workloads. A multi-user deployment would normally move central records to a managed database such as PostgreSQL and place uploaded images in private object storage.

7. How the phone and server communicate

HTTP and JSON

HTTP is the request-and-response system used by web services. JSON is the plain-text data format carried in those requests. The app's API client converts TypeScript objects into JSON and sends them to FastAPI.

```
POST /observations
Content-Type: application/json

{
  "crop": "cassava",
  "diagnosis_id": "cassava_mosaic_disease",
  "confidence": 0.92,
  "sync_status": "PENDING"
}
```

Why 10.0.2.2 is used during development

Inside the Android emulator, **10.0.2.2** is a special address that points back to the development computer. That is why the prototype API address is **http://10.0.2.2:8000**. A physical phone instead needs a reachable network address.

Production networking

Prototype	Production direction
HTTP development address	HTTPS API domain with a valid certificate
Cleartext traffic allowed	Cleartext disabled
CORS allows every origin	Restricted origins and explicit policy
No user authentication	Authenticated and authorized requests
Editable API address	Environment-specific configuration
Local image URI sent as text	Secure image upload or local-only photo policy

Security principle
Never put backend passwords or private API keys inside the Android app. Installed applications can be inspected. Secrets belong on the server or in a protected deployment secret manager.

8. Development, testing, and deployment tools

Tool	Beginner explanation
Git	Tracks source-code history so changes can be reviewed and reversed.
npm	Installs the mobile libraries listed in package.json.
Expo CLI	Starts development, opens the Android project, and checks the Expo setup.
Metro	Builds and serves the JavaScript bundle during development.
Android Studio emulator	Provides a virtual Android phone for testing.
ADB	Connects development tools to an emulator or physical Android device.
TypeScript compiler	Checks mobile data types without creating a build.
ESLint	Finds suspicious or inconsistent code patterns.
Expo Doctor	Checks dependency compatibility and project configuration.
Pytest	Runs automated backend tests.
HTTPX	Helps backend tests send requests to the FastAPI application.

Common validation commands

```
# Mobile
cd mobile
npm run typecheck
npm run lint
npm run validate:guidance
npm run doctor

# Backend
cd backend
pytest -q
```

What gets deployed

Part	Deployment artifact
Android app	A signed Android App Bundle (.aab) for Google Play, or an APK for controlled installation.
Backend	The Python/FastAPI service running on a secured server or container platform.
Central data	A production database, plus private object storage if photos are uploaded.
AI model	A reviewed .tflite model bundled with or securely delivered to the app.
Guidance	A reviewed, versioned offline catalog bundled with the app and optionally updated from the server.

9. One diagnosis from start to finish

1	Photo	Expo Image Picker opens the camera or gallery.
2	Private copy	Expo File System copies the image into the app's document storage.
3	Classification	The mock classifier currently returns a fixed result; MobileNetV3 will later analyze the pixels.
4	Guidance lookup	The diagnosis ID selects the matching record from guidance.json.
5	Result	React Native displays the image, confidence, symptoms, and actions.
6	Local save	SQLite stores the observation as PENDING.
7	Sync	The app turns pending records into JSON and sends them over HTTP.
8	Validation	FastAPI checks the data and writes it to the backend database.
9	Confirmation	The phone marks the local record SYNCED after server acknowledgment.

Mini glossary

Term	Meaning
Framework	A prepared structure that helps developers build an application.
Library	Reusable code that provides a particular capability.
API	A defined way for two programs to exchange requests and responses.
Frontend	The part of the system the user sees and touches.
Backend	The server-side program that receives and stores shared data.
Database	An organized system for storing and retrieving records.
Runtime	Software that executes a program, such as Hermes or Python.
Bundle	Application code and assets packaged together for execution or installation.
Inference	Using a trained AI model to make a prediction from new input.
Offline-first	Saving locally first so the main workflow still functions without internet.

The key design idea
LimaDRC separates responsibilities: the model identifies a likely condition, the guidance catalog supplies reviewed information, SQLite protects offline work, and the backend synchronizes shared records.

10. Current status and next production steps

Area	Current state	Next step
Android interface	Implemented with Expo/React Native	Complete device testing and accessibility review
Camera/gallery	Implemented	Test permissions and photo persistence across target Android versions
Offline database	Implemented with SQLite	Add migrations, backup/privacy rules, and failure testing
Guidance	Bundled JSON catalog implemented	Expert review, localization, sources, and controlled updates
Classifier	Mock result	Train, evaluate, calibrate, export, and integrate MobileNetV3
Synchronization	Prototype queue and API implemented	Authentication, HTTPS, conflict policy, retries, and image handling
Backend storage	Prototype SQLite	Choose managed production database and deployment platform
Security	Development configuration	Threat review, restricted CORS, secret management, and data protection

Source files used for this guide

- **mobile/package.json** - exact mobile dependencies and development scripts.
- **mobile/app.json** - Expo and Android configuration.
- **mobile/src/app/** - screens and navigation.
- **mobile/src/storage/** - SQLite tables and private photo persistence.
- **mobile/src/services/** - HTTP API client and synchronization queue.
- **mobile/src/ml/** - classifier contract, mock classifier, labels, and offline guidance.
- **backend/main.py** - FastAPI endpoints and backend SQLite operations.
- **backend/requirements.txt** - backend library versions.

Quick summary

The app's core is **React Native + Expo + TypeScript**. It stores work offline with **SQLite**, keeps photos with **Expo File System**, and exchanges **JSON over HTTP** with a **Python/FastAPI** backend. Diagnosis guidance is a local **JSON catalog**. **MobileNetV3 exported to TensorFlow Lite** is planned to replace the current mock classifier.

In one sentence

This stack is designed to give the farmer a useful Android workflow even when the network is unreliable, while leaving a clear path to on-device crop recognition and secure production synchronization.